Implementación Catálogo de Metadatos

Metadata Catalog Implementation

Niels Alexander Cadengo Rodriguez
Instituto Tecnológico de Aguascalientes
Dep. Sistemas y Computación
Aguascalientes, México
17151310@aguascalientes.tecnm.mx

MSc. Edgar Oswaldo Díaz
Instituto Nacional de Estadística y Geografía (INEGI) Dep.
Laboratorio de Ciencias de datos y Métodos Modernos de Producción de Información
Aguascalientes, México
Oswaldo.Diaz@inegi.org.mx

Resumo — **En el presente documento se observa el trabajo realizado a lo largo de estos 6 meses en los cuales se observará lo relacionado a la implementación de un catálogo de metadatos con la aplicación apache atlas, que permite que desde sitios remotos los usuarios encuentren, accedan y consulten sus recursos disponibles, básicamente capas de datos geográficos, a través de los llamados metadatos.**

El Servicio de Catálogo facilita la publicación y búsqueda de la descripción de los datos (metadatos) y de los servicios Web, a través de un protocolo estándar de comunicación interoperable que vincula las peticiones entre el usuario y el servidor de metadatos, con lo que el usuario recibe una respuesta acerca de los registros de metadatos de los datos que le interesan.

Se realizó una serie de consultas al lago de datos de la red de INEGI para extraer así los metadatos y estos poder ser publicados en el catálogo de metadatos y graficados en el lineage de datos de Apache Atlas.

Palabras Clave - metadatos; catalogo metadatos; big data; contenedores; virtualizacion; hypervisor; rhel; protocolo; json;

Abstract — **This document shows the work carried out during these 6 months in which we will observe the implementation of a metadata catalog with the apache atlas application, which allows users from remote sites to find, access and consult their available resources, basically geographic data layers, through the so-called metadata.**

The Catalog Service facilitates the publication and search of data description (metadata) and Web services, through a standard interoperable communication protocol that links requests between the user and the metadata server, so that the user receives a response about the metadata records of the data he is interested in.

A series of queries were made to the INEGI network data lake to extract the metadata to be published in the metadata catalog and plotted in the Apache Atlas data lineage.

Keywords - metadata; metadata catalog; big data; containers; virtualization; hypervisor; rhel; protocol; json;

I. Introducción

A. Metadatos

Del griego meta, "después de" y de "data" plural del latín datum , "datos" , literalmente significa "más allá de los datos", aludiendo a datos que describen otros datos. En concreto, el uso del término desde un enfoque informático toma como punto de partida esta concepción genérica, tanto de forma individual como cuando un grupo de metadatos describe a un grupo de datos o recursos.

Los metadatos se caracterizan por:

- Ser datos altamente estructurados que describen características de los datos, como el contenido, calidad, información y otras circunstancias o atributos.
- Presentan diferenciaciones que dependerán, en última instancia, de las reglas incluidas en las aplicaciones para determinar la estructura interna de los esquemas de datos.
- Pueden clasificarse en función de distintos criterios, como su contenido, variabilidad o función.

En el actual contexto de Big Data, Internet de las cosas y cloud computing los metadatos han adquirido una relevancia inédita. Frente a ingentes cantidades de información que crecen de forma exponencial, la gestión de los metadatos puede ser de gran ayuda para ganar en eficiencia operativa y/o tomar mejores decisiones para la obtención de ventajas competitivas.

En el entorno organizacional los metadatos juegan un importante papel como marco de referencia, en línea con una visión genérica que Forrester Research resume definiéndolos como "la información que describe o proporciona el contexto para los datos, contenidos, procesos de negocio, servicios, reglas de negocio y políticas de apoyo a los sistemas de información de una organización".

Éstas son algunas de las funciones más ventajosas de una buena administración de los metadatos a la hora de mejorar los procesos de gestión y gobierno de datos:

- Facilita búsquedas y análisis: Los metadatos ayudan a buscar y ubicar datos. Una buena gestión de metadatos también facilita el análisis del curso de los datos desde la fuente, facilitando la auto documentación, así como funciones de transformación, análisis y reporting.
- Facilita la estandarización: Al eliminar errores e inconsistencias, la estandarización de datos mejora la calidad de los mismos a lo largo de su ciclo de vida. Con la gestión de los metadatos en un repositorio centralizado conseguiremos una visión más completa del ciclo de vida del dato, desde que se crea hasta que se consume, además de ventajas en el control de procesos.
- Ayuda a la integración: En la integración híbrida los metadatos son claves. Sirviéndonos de un repositorio centralizado de metadatos para el uso compartido entre los usuarios de TI y de negocio facilitaremos la gobernanza, así como una aplicación de las mejores prácticas por parte de aquellos. De gran utilidad en estructuras híbridas para mejorar la gestión de datos de forma integrada.
- Gestión del cambio: Sobre todo, en entornos complejos, pues la gestión de metadatos proporciona la visibilidad y el control necesarios para hacerlo en un entorno de integración de datos empresariales. A través de una automatización de los análisis de impacto detectaremos los cambios en las aplicaciones y podremos intervenir para subsanar conflictos.
- Más seguridad: En caso de haber cambios, una adecuada gestión de metadatos protegerá los datos críticos del negocio y, en general, facilitará el cumplimiento de la normativa.
- Mejora los informes: Gracias a esa facilidad de intervención datos serán de calidad y, los procesos no presentarán incidencias y, lógicamente, el reporting ganará en confiabilidad. En general, una correcta gestión de metadatos permitirá entregar datos seguros y confiables.
- Desarrollos más ágiles: Un acceso inteligente a los metadatos, por ejemplo en un entorno híbrido integrado, aumenta la productividad de los desarrolladores y reduce el plazo de suministro de la conectividad. Ello se traduce en una rebaja del coste del cambio entre las diferentes plataformas.
- Mejor gobernanza de datos: Los metadatos gestionados en un entorno estandarizado mediante un repositorio centralizado son esenciales para implementar un exitoso programa de gobierno de datos. Entre otras ventajas, la gestión de los metadatos aumenta la visibilidad de las distintas ejecuciones de patrones y gestión de diferentes fuentes de datos, propiciando una gobernanza centralizada, así como las mejores prácticas. [1]

B. *Catálogo de metadatos*

Un catálogo de datos es un inventario de los activos de datos de una empresa para que los usuarios puedan encontrar la información que necesitan rápidamente. El catálogo consiste principalmente en metadatos que brindan información básica sobre otros datos y describen su contenido. Combinado con la gestión de datos y las herramientas de búsqueda.

Dentro del Big Data, los catálogos de metadatos son un componente clave en la gestión de datos. Las personas que trabajan con datos utilizan catálogos de datos para buscar los activos de datos requeridos de la totalidad de las fuentes de una organización, que pueden estar dispersas y ser difíciles de explorar. Las implementaciones exitosas de catálogos de metadatos pueden marcar una gran diferencia en la velocidad y la calidad del análisis de datos porque ayudan a los usuarios a encontrar los datos que necesitan rápidamente.

Los catálogos de metadatos ofrecen una serie de beneficios para la organización. En primer lugar, un catálogo de datos puede brindar a los usuarios todas las fuentes correctas, en el formato correcto, en la vista correcta, en el momento correcto, con el nivel de control correcto. Los catálogos de datos aseguran que toda la información que tiene en todas sus diferentes fuentes en un contexto de múltiples nubes se pueda encontrar y se pueda consumir de inmediato, lo cual significará que los usuarios pueden construir e implementar modelos en un contexto en tiempo real.

Además de ofrecer el contexto a los analistas de datos que requieren utilizar los datos con fines comerciales, los catálogos de datos también permiten automatizar la gestión de metadatos.[2]

C. *Apache atlas*

Apache Atlas es una herramienta enfocada a resolver la mayoría de los principales retos del Data Governance. Es una herramienta open-source, con licencia Apache 2.0, para la gobernanza del dato la cual permite la integración con todo el ecosistema de datos de las empresas.

Atlas permite crear un almacén de metadatos centralizado para saber dónde encontrar un conjunto de datos dentro de la empresa (Data Discovery), permite saber qué cambios ha sufrido y que transformaciones se le han realizado a los datos originales a lo largo del tiempo (Data Lineage) y centraliza la seguridad para saber quién puede acceder a esos datos y quién puede modificarlos (Data Security) apoyándose en Apache Ranger.

Data Discovery

Atlas permite a los usuarios definir un modelo para los objetos de metadatos que se desean administrar. Todos los objetos de metadatos administrados por atlas que están listos para ser utilizados se modelan usando “Tipos” y se representan como “Entidades”. Para facilitar la utilización de la herramienta

Atlas permite crear glosarios donde se puede relacionar términos y categorías con los datos para que sea más fácil de encontrar.

Data Lineage

Apache Atlas permite hacer un seguimiento de las transformaciones que han sufrido los datos originales y en qué datos derivados han terminado para poder tener un control total sobre ellos.

Data Security

Atlas permite autentificar personal con los siguientes métodos: LDAP,File y Kerberos. Se pueden utilizar varios a la vez y en caso de que falle uno se pueda tener una alternativa. Además permite la creación de usuarios con distintos permisos de forma básica y a estos usuarios se les puede modificar la política de autorizaciones apoyándose en Apache Ranger.[3]

D. MinIO S3

MinIO es un software que ofrece almacenamiento de Objetos enfocado en al Rendimiento y liberado bajo la Licencia Apache v2.0 con un API compatible con Amazon S3.

Es un servidor de almacenamiento de objetos distribuidos de alto rendimiento, que está diseñado para la infraestructura de nube privada a gran escala. Minio agrega los volúmenes persistentes (PVs) en el almacenamiento de objetos distribuidos escalable, utilizando las API REST de Amazon S3.

Características de minio s3

- Es más adecuado para almacenar datos no estructurados como, por ejemplo, fotos, vídeos, archivos de registro, copias de seguridad, máquinas virtuales e imágenes de contenedor.
- Minio permite la modalidad de pasarela del almacenamiento conectado a la red (NAS) distribuido y autónomo.
- MinIO es ideal para despliegues en Cloud privadas con altos requisitos en seguridad y alta disponibilidad, y para un amplio rango de workloads (sistema de ficheros distribuidos, ML, analítica, entre otros).
- Uno de sus focos es el rendimiento, donde MinIO se posiciona como el servidor de almacenamiento de objetos más rápido, con velocidades de lectura y escritura de 183 Gb/s y 171 Gb/s en hardware estándar.
- MinIO puede funcionar como la capa de almacenamiento primario para soluciones basadas en Spark, Presto, Tensorflow, H20.ai además de poder reemplazar a Hadoop HDFS.
- MinIO nació con un foco en un despliegue Cloud por lo que está pensado para funcionar en entornos contenerizados y multi-tenancy. Además su integración con Kubernetes es total.[4]

E. Postgres SQL

PostgreSQL es un sistema de bases de datos de código abierto, altamente estable, que proporciona soporte a diferentes funciones de SQL, como claves foráneas, subconsultas, disparadores y diferentes tipos y funciones definidas por el usuario. Además, aumenta el lenguaje SQL ofreciendo varias funciones que escalan y reservan meticulosamente las cargas de trabajo de datos. Se utiliza principalmente para almacenar datos para muchas aplicaciones móviles, web, geoespaciales y de análisis.

Se trata de un sistema de código abierto y además gratuito, y su desarrollo es llevado adelante por una gran comunidad de colaboradores de todo el mundo que día a día aportan para hacer de este sistema una de las opciones más sólidas a nivel de bases de datos.

El servidor PostgreSQL tiene una estructura sencilla, que consiste en una memoria compartida, procesos en segundo plano y una estructura de directorio de datos. En esta sección, hablaremos de cada componente y de cómo interactúan bufferentre sí. A continuación se muestra una ilustración de la arquitectura de PostgreSQL. Inicialmente, el cliente envía una petición al servidor. A continuación, el servidor PostgreSQL procesa los datos utilizando bufferes compartidos y procesos en segundo plano. El archivo físico del servidor de bases de datos PostgreSQL se almacena en el directorio de datos.[5]

F. Hive

Hive es una tecnología distribuida diseñada y construida sobre Hadoop. Permite hacer consultas y analizar grandes cantidades de datos almacenados en HDFS, en la escala de petabytes. Tiene un lenguaje de consulta llamado HiveQL o HQL que internamente transforma las consultas SQL en trabajos MapReduce que ejecutan en Hadoop. El lenguaje de consulta HQL es un dialecto de SQL, que no sigue el estándar ANSI SQL, sin embargo es muy similar.

Los datos gestionados por Hive son datos estructurados almacenados en HDFS. Así, optimiza de forma automática el plan de ejecución y usa particionado de tablas en determinadas consultas. También soporta diferentes formatos de ficheros, codificaciones y fuentes de datos como HBase.

Una consulta típica en Hive ejecuta en varios data nodes en paralelo, con trabajos MapReduce asociados. Estas operaciones son de tipo batch, por lo que la latencia es más alta que en otros tipos de bases de datos. Además, hay que considerar el retardo producido por la inicialización de los trabajos, sobre todo en el caso de consultar pequeños datasets.

Hive incorpora Beeline, un cliente basado en JDBC para hacer consultas por línea de comandos contra el componente HiveServer, sin necesitar las dependencias de Hive. Por otro lado, también incorpora Hive CLI, un cliente basado en Apache Thrift, que usa los mismos drivers que Hive.

Estructura de datos en Hive

Hive proporciona una estructura basada en tablas sobre HDFS. Soporta tres tipos de estructuras: Tablas, particiones y

buckets. Las tablas se corresponden con directorios de HDFS, las particiones son las divisiones de las tablas y los buckets son las divisiones de las particiones.

Hive permite crear tablas externas, similares a las tablas en una Base de datos, pero a la que se le proporciona una ubicación.[6]

Hive Server

HiveServer2 (HS2) es un servicio que permite a los clientes ejecutar consultas en Hive. HiveServer2 es el sucesor de HiveServer1, que ha quedado obsoleto. HS2 admite autenticación y concurrencia de varios clientes. Está diseñado para proporcionar un mejor soporte para clientes API abiertos como JDBC y ODBC.

HS2 es un proceso único que se ejecuta como un servicio compuesto, que incluye el servicio Hive basado en Thrift (TCP o HTTP) y un servidor web Jetty para la interfaz de usuario web.[7]

Hive Metastore

Hive Metastore (HMS) es un repositorio central de metadatos para las tablas y particiones de Hive en una base de datos relacional, y proporciona a los clientes (incluidos Hive, Impala y Spark) acceso a esta información mediante la API del servicio Metastore.

Reside en una base de datos relacional como MySQL, PostgreSQL o Apache Derby (base de datos interna), donde persiste la información. Mantiene un seguimiento de los metadatos, las tablas y sus tipos mediante Hive DDL (Data Definition Language).[8]

G. Virtualizacion

La virtualización es una tecnología que permite crear servicios de TI útiles, con recursos que están tradicionalmente limitados al hardware. Gracias a que distribuye las funciones de una máquina física entre varios usuarios o entornos, posibilita el uso de toda la capacidad de la máquina.

Las máquinas virtuales (VM) son entornos virtuales que simulan una computación física en forma de software. Normalmente incluyen varios archivos que contienen la configuración de la máquina virtual, el almacenamiento del disco duro virtual y algunas instantáneas de la máquina virtual que conservan su estado en un momento determinado.

Hipervisor

El software denominado hipervisor separa los recursos físicos de los entornos virtuales que los necesitan. Los hipervisores pueden controlar un sistema operativo (como una computadora portátil) o instalarse directamente en el hardware (como un servidor), que es la forma en que la mayoría de las empresas implementan la virtualización. Los hipervisores toman los recursos físicos y los dividen de manera tal que los entornos virtuales puedan usarlos.

Los recursos se dividen según las necesidades, desde el entorno físico hasta los numerosos entornos virtuales. Los usuarios interactúan con la informática y la ponen en funcionamiento dentro del entorno virtual (generalmente denominado máquina guest o máquina virtual). La máquina virtual funciona como un archivo de datos único; por eso, tal como ocurre con cualquier archivo digital, es posible trasladarla de una computadora a otra, abrirla en cualquiera de ellas, y tener la tranquilidad de que funcionará de la misma forma.

Cuando el entorno virtual se está ejecutando, y un usuario o programa emite una instrucción que requiere recursos adicionales del entorno físico, el hipervisor transmite la solicitud al sistema físico y almacena los cambios en la caché. Todo esto sucede prácticamente a la misma velocidad que habría si este proceso se realizara dentro de la máquina física.[9]

H. Sistema operativo RHEL

Red Hat Enterprise Linux es un sistema operativo Linux para empresas, el cual está certificado en cientos de nubes y con miles de proveedores. Proporciona una base sólida para todos los entornos y las herramientas necesarios para prestar servicios y ejecutar cargas de trabajo con mayor rapidez para cualquier aplicación. Además, no solo reduce los inconvenientes y los costos asociados a la implementación, sino que también agiliza la obtención de resultados para las cargas de trabajo más importantes de la empresa, lo cual permite que los equipos de desarrollo y operaciones generen innovaciones juntas en cualquier entorno.

Red Hat Enterprise Linux no solo le ofrece un sistema operativo, sino que también lo conecta con el amplio ecosistema de partners de nube, de software y de hardware de Red Hat, e incluye un servicio de soporte permanente.

Red Hat Enterprise Linux se puede optimizar para ejecutarse en los servidores o las estaciones de trabajo de alto rendimiento. Además, admite diversas arquitecturas de hardware, como x86, ARM, IBM Power, IBM Z e IBM LinuxONE. Nuestra estrecha colaboración con las comunidades upstream y los partners de hardware nos permite ofrecerle una plataforma confiable para varios casos prácticos y un entorno de aplicaciones uniforme en las implementaciones virtuales, físicas y de nube.[10]

I. Contenerización

La contenerización es el empaquetado de código de software con solo las bibliotecas de sistema operativo (SO) y las dependencias necesarias para ejecutar el código para crear un único ejecutable ligero, denominado contenedor, que se ejecuta de manera compatible en cualquier infraestructura. Más portátil y eficiente en recursos que las máquinas virtuales (VM), los contenedores se han convertido en las unidades de cálculo de facto de las aplicaciones nativas en la nube modernas.

La contenerización permite a los desarrolladores crear y desplegar aplicaciones de forma más rápida y segura. Con los métodos tradicionales, el código se desarrolla en un entorno informático específico que, cuando se transfiere a una nueva ubicación, a menudo provoca errores. La contenerización elimina este problema ya que agrupa el código de aplicación con

los archivos de configuración, las bibliotecas y las dependencias correspondientes que se necesitan para su ejecución. Este paquete único de software o "contenedor" se abstrae del sistema operativo del host, y por lo tanto, es independiente y portátil, capaz de ejecutarse en cualquier plataforma o nube, sin ningún problema.

Ventajas de la contenerización

- Portabilidad: un contenedor crea un paquete ejecutable de software que se abstrae (no vinculado ni dependiente) del sistema operativo del host, y por lo tanto, es portátil y puede ejecutarse de forma uniforme y coherente en cualquier plataforma o nube.
- Agilidad: el Docker Engine de código abierto para ejecutar contenedores inició el estándar del sector para contenedores con herramientas de desarrollador simples y un método de empaquetado universal que funciona en sistemas operativos Linux y Windows. El ecosistema de contenedores se ha reorientado a los motores gestionados por Open Container Initiative (OCI). Los desarrolladores de software pueden seguir utilizando herramientas y procesos ágiles o DevOps para acelerar el desarrollo y mejorar las aplicaciones.
- Velocidad: los contenedores a menudo se denominan "ligeros", lo que significa que comparten el kernel del sistema operativo (SO) de la máquina sin sufrir esta sobrecarga adicional. Esto no solo aumenta la eficiencia del servidor, sino que también reduce los costes de servidor y licencia y acelera los tiempos de inicio, ya que no es necesario arrancar ningún sistema operativo.
- Aislamiento de errores: cada aplicación en contenedor está aislada y opera de forma independiente de las otras. El fallo de un contenedor no afecta a la continuidad de las operaciones de otros contenedores. Los equipos de desarrollo pueden identificar y corregir cualquier problema técnico dentro de un contenedor sin ningún tiempo de inactividad en otros contenedores. Además, el motor de contenedores puede aprovechar cualquier técnica de aislamiento de seguridad del sistema operativo, como el control de acceso de SELinux, para aislar errores dentro de los contenedores.
- Eficiencia: el software que se ejecuta en entornos contenerizados comparte el kernel del sistema operativo de la máquina y las capas de aplicación dentro de un contenedor se pueden compartir entre los distintos contenedores. De esta manera, los contenedores son inherentemente más pequeños en capacidad que una máquina virtual y requieren menos tiempo de inicio, lo que permite ejecutar muchos más contenedores en la misma capacidad de cálculo que una sola máquina virtual. Esto genera una mayor eficiencia del servidor, reduciendo los costes de servidor y licencia.
- Gestión sencilla: una plataforma de orquestación de contenedores automatiza la instalación, el escalado y la gestión de cargas de trabajo y servicios contenerizados. Las plataformas de orquestación de contenedores facilitan algunas tareas de gestión como escalar aplicaciones contenerizadas, desplegar nuevas versiones de aplicaciones y proporcionar supervisión, registro y depuración, entre otras funciones. Kubernetes, tal vez el sistema de orquestación de contenedores más popular, es una tecnología de código abierto (creada originalmente por Google, basada en su proyecto interno llamado Borg) que automatiza las funciones de contenedor de Linux en su origen. Kubernetes funciona con muchos motores de contenedor, como Docker, pero también funciona con cualquier sistema de contenedores que se ajuste a los estándares de Open Container Initiative (OCI) para formatos de imagen de contenedor y tiempos de ejecución.
- Seguridad: el aislamiento de las aplicaciones como contenedores impide de forma inherente que la invasión de código malicioso afecte a otros contenedores o al sistema host. Además, se pueden definir permisos de seguridad para bloquear automáticamente los componentes no deseados y que no entren en los contenedores ni limiten las comunicaciones con recursos innecesarios.[11]

J. Podman

Podman (Pod Manager) es un motor de contenedores lanzado por primera vez en febrero de 2018 por la empresa de software estadounidense Red Hat, conocida principalmente por sus soluciones empresariales de alto rendimiento para diferentes proyectos de código abierto, como Red Hat Enterprise Linux (RHEL) u OpenStack. Basándose en la experiencia de Docker, en un principio estaba previsto que Podman no fuese un motor independiente, sino una herramienta de depuración sencilla para CRI-O.

Podman funciona en las principales distribuciones de Linux, como Ubuntu, Fedora, CentOS, Debian y RHEL, así como en Raspbian (sistema operativo Raspberry Pi). En la mayoría de los casos, el motor se puede instalar directamente desde la configuración del paquete del sistema correspondiente.

Además de renunciar a un daemon central, otra de las características más destacadas de Podman son los denominados pods. Estos pods, inspirados en el concepto de los pods de Kubernetes, son la fusión de varios contenedores en un namespace común de Linux que comparten recursos concretos. De esta manera, se pueden combinar una amplia variedad de aplicaciones virtualizadas.[12]

II. Formato General

Estas actividades realizadas se desarrollaron de manera satisfactoria debido a que la propuesta de la aplicación fue platicada con los colaboradores y no se requirió mucho tiempo para poder concluir con la estrategia adecuada para implementar el catálogo de datos.

Un catálogo de metadatos se puede crear de varias maneras, pero para garantizar la implementación exitosa de un catálogo de datos eficiente, son necesarios los siguientes factores.

El catálogo de metadatos sirve como un único lugar de confianza para los datos. Los conectores mapean los conjuntos de datos físicos en su base de datos; por lo tanto, es importante contar con una amplia gama de conectores para reforzar el catálogo de datos.

Dado que los metadatos se pueden recopilar de múltiples fuentes, como Salesforce, consultas SQL, inteligencia de negocio o herramientas de integración de datos, también es importante conservar estos datos. La validación y la certificación son procesos importantes que mejoran la eficiencia de un catálogo de datos y hacen que el gobierno de datos sea un proceso sostenible.

Para esto se hizo una serie de implementaciones de distintas aplicaciones para poder llegar a la solución adecuada.

Partimos del lago de datos que en este caso es un almacenamiento en la nube de tipo objeto (minIO s3), para esto tuvimos que hacer una implementación de la aplicación para poder tener acceso a los datos dentro de la red de INEGI a los cuales se le extraerán los metadatos y estos se guardaran en el metastore que está compuesto por dos aplicaciones, en este caso hive que tendrá la función de motor de consultas hacia el s3 y a la vez el servicio de metastore donde con ayuda de la aplicación y bases de datos de postgreSQL, podra almacenar las referencias de los metadatos de los datos de minIO, teniendo así un tipo índice de metadatos para así poder encontrar la información de forma mas rápida, de esta forma esos metadatos podran ser importados con ayuda de hive hacia Apache Atlas y así logrando ser representados en forma de grafos y con todas la relaciones existentes de esas entidades creadas en Apache Atlas.

Para poder iniciar con el procedimiento de implementación del catálogo de datos, tuvimos que adaptarnos a los sistemas que se utilizan en el laboratorio de ciencia de datos, en este caso tuvimos que hacer una instalación de un sistema red hat entreprise linux (RHEL) virtualizado, el cual nos va a permitir hacer uso de programas como Podman que nos permitirá trabajar dentro del ambiente de contenedores, ya que son las tecnologías con las que trabaja el LCID.

Este sistema RHEL se virtualizo con la ayuda de un hypervisor, utilizamos la aplicación VMware Workstation que no permite crear máquinas virtuales con los mismos recursos de nuestra maquina host, emulando recursos que serán asignados y visto como “reales en la máquina virtual”. Este sistema nos permitirá hacer uso de la aplicación podman que ya se encuentra en el mismo kernel de linux y esta nos ayudara a la creación y gestión de contenedores, permitiendo correr imágenes de los sistemas y/o aplicaciones que necesitamos.

A. *Creacion maquina virtual con VMware Workstation*

VMware Workstation es una línea de productos de Hipervisor de escritorio que permiten a los usuarios ejecutar máquinas virtuales, contenedores y clústeres de Kubernetes.

Los productos VMware Workstation utilizan funciones especiales en CPU x86 de 64 bits modernas para crear máquinas virtuales seguras y completamente aisladas que encapsulan un sistema operativo y sus aplicaciones. La capa de virtualización de VMware asigna los recursos de hardware físicos a los recursos virtuales de la máquina virtual, por lo que cada máquina virtual cuenta con una CPU, una memoria, unos discos y unos dispositivos de E/S propios, y equivale en su totalidad a una máquina x86 convencional.

VWware nos permitirá crear una máquina virtual con la cual se trabajara sobre un sistema operativo Linux RHEL para la implementación de las herramientas necesarias.

Pasos para crear máquina virtual:

1.- Instalar VMware Workstation pro y crear máquina virtual con sistema RHEL .

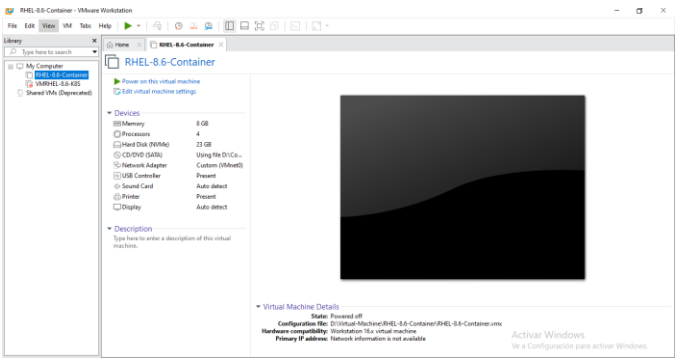

Figura 1.- Pantalla inicio Vmware Workstation

2.- Configurar sistema RHEL.

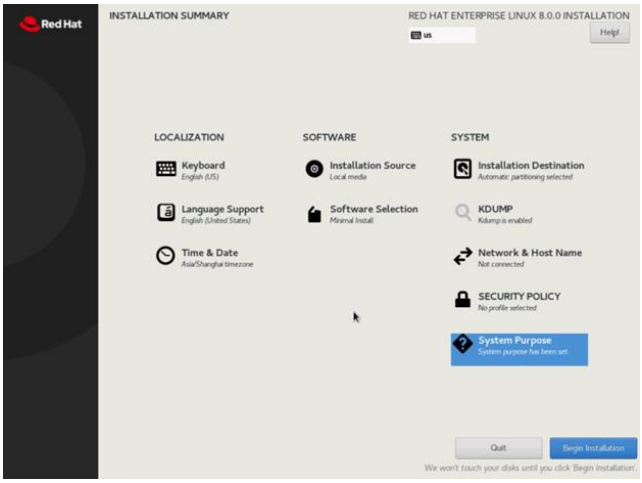

Figura 2.- Pantalla instalación RHEL

3.- Una vez instalado el sistema operativo ya podremos acceder y hacer uso del mismo.

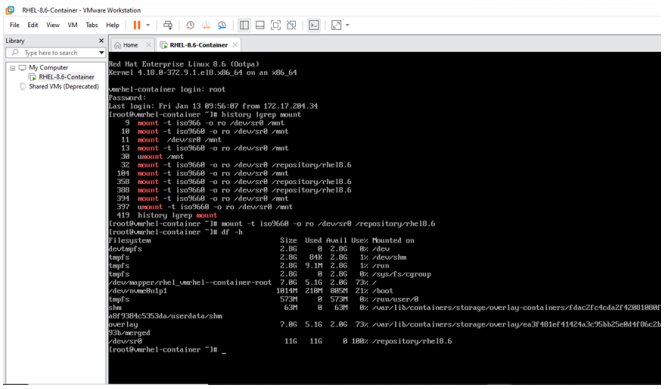

Figura 3.- Ejecución sistema RHEL

4.- Ya teniendo el sistela RHEL podremos hacer uso del gestor de contenedores (podman) que ya viene integrado en el kernel del sistema.

Preparando imagen contenedor Minio s3.

5.- Con ayuda del comando pull podremos descargar la imagen del sitio oficial de minIO s3 para poder preparar el contenedor posteriormente poder ejecutarla con sus respectivas configuraciones.

podman pull quay.io/minio/minio

Para que minio funcione y pueda almacenar contenido debemos crear un volumen y asignarlo al contenedor.

6.- Con el comando run crearemos el contenedor, asignamos los parametros correspondientes.

podman run -it --name minio-server --hostname minio-server -p 9000:9000 -p 9001:9001 --privileged -v /residencias/minio/data:/data -e MINIO_REGION="us-east-1" -e MINIO_ROOT_USER="ROOTNAME" -e MINIO_ROOT_PASSWORD="CHANGEME123" minio/minio server /data --console-address ":9001"

Accedemos a la web por el puerto 9001 para ingresar al login minio.

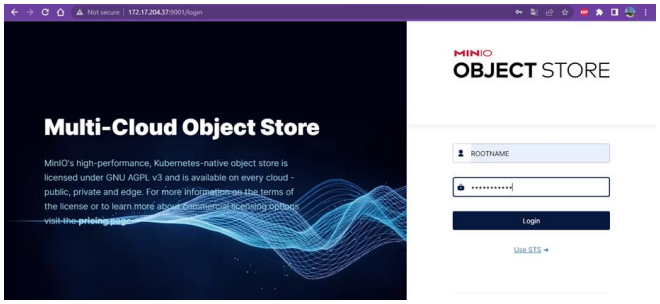

Figura 4.- Interfaz web de minIO

7.- Creamos un bucket de almacenamiento el cual se hará referencia para que se pueda almacenar los metadatos de bases de datos, tablas y/o registros.

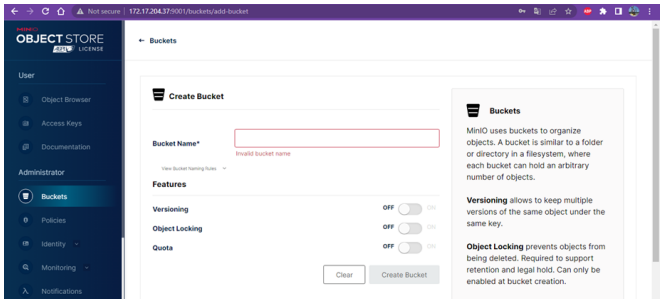

Figura 5.- Interfaz crear bucket

Ya creado el bucket podemos almacenar cualquier tipo de objeto.

Preparando postgreSQL para almacenamiento de metadatos

8.- Descargamos imagen para contenedor.

podman pull docker.io/dockyuki1992/postgres14.4

9.- Creamos contenedor postgres con comando run.

#podman run -d --privileged --name postgres --add-host postgres:<addr> -e POSTGRES_PASSWORD=test -e POSTGRES_USER=test -p 5432:5432 docker.io/dockyuki1992/postgres14.4

10.- Acceder al contenedor postgres con el comando exec.

podman exec -it postgres /bin/bash

El comando psql ejecuta un front-end basado en la terminal de postgreSQL, e permite escribir consultas de forma interactiva, enviarlas a postgreSQL y ver los resultados de la consulta.

11.- Creamos una base de datos que será la que se hará referencia para el metastore de hive y ahí pueda crear su esquema de datos.

CREATE DATABASE test99;

12.- Con el comando “\l” podemos ver todas bases de datos existentes.

```
[root@vmrhel-container ~]# podman exec -it postgres bash
root@021582ebf6ea:/# psql -U test
psql (14.4 (Debian 14.4-1.pgdg110+1))
Type "help" for help.

test=# \l
                                  List of databases
    Name     | Owner | Encoding |  Collate   |   Ctype    | Access privileges
-------------+-------+----------+------------+------------+-------------------
 metastoredb | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 postgres    | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 template0   | test  | UTF8     | en_US.utf8 | en_US.utf8 | =c/test          +
             |       |          |            |            | test=CTc/test
 template1   | test  | UTF8     | en_US.utf8 | en_US.utf8 | =c/test          +
             |       |          |            |            | test=CTc/test
 test        | test  | UTF8     | en_US.utf8 | en_US.utf8 |
 test99      | test  | UTF8     | en_US.utf8 | en_US.utf8 |
(6 rows)
```

Figura 6.- Bases de datos existentes

Preparando Hive metastore y hive server.

13.- Descargamos imagen contenedor hive.

podman pull docker.io/starburstdata/hive

Con esta imagen levantaremos un contenedor hive-metastore y otro hive-server.

14.- Creamos los siguientes archivos con los parámetros de conexión para postgreSQL y Minio en cada contenedor respectivamente.

Configuración hive metastore

Dentro de una carpeta hive-metastore crearemos los archivos de configuración para posteriormente hacer el remplazo de archivos agregándolos como volumen.

Con ayuda del comando nano podremos crear los archivos .xml de configuracion.

nano hive-site-metastore.xml

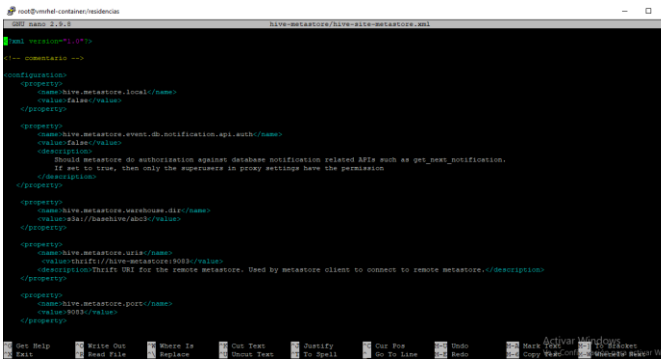

Figura 7.- Configuración hive site metastore

15.- Creamos archivo de configuración core-site.xml para asignar memoria.

nano core-site.xml

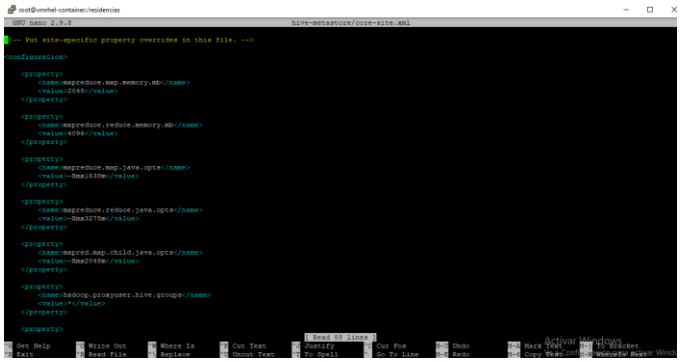

Figura 8.- Configuración core site

16.- Creamos contenedor hive-metastore.

podman run -it --name hive-metastore --hostname hive-metastore -p 9083:9083 --privileged -v /residencias/hive-metastore/hive-site-metastore.xml:/opt/apache-hive-3.1.2-bin/conf/hive-site.xml -v /residencias/hive-metastore/core-site.xml:/opt/hadoop-3.3.1/etc/hadoop/core-site.xml -v /residencias/hive-metastore/hosts:/etc/hosts starburstdata/hive:3.1.2-e.15-x64 /bin/bash

17.- Accedemos al contenedor hive-metastore y ejecutaremos en segundo plano con ayuda del comando nohup el servicio de metastore para iniciar el proceso.

nohup ./hive --service metastore &

```
[hive@hive-metastore bin]$
[hive@hive-metastore bin]$ cd /opt/apache-hive-3.1.2-bin/bin
[hive@hive-metastore bin]$ nohup ./hive --service metastore &
[1] 45
[hive@hive-metastore bin]$ nohup: ignoring input and appending output to 'nohup.out'
```

Figura 9.- Servicio metastore segundo plano

Configuración hive server

De igual forma dentro de una carpeta hive-server crearemos los archivos de configuración para hive server y posteriormente hacer el remplazo de archivos agregándolos como volumen.

18.- Con ayuda del comando nano podremos crear los archivos .xml de configuracion.

nano hive-site-server.xml

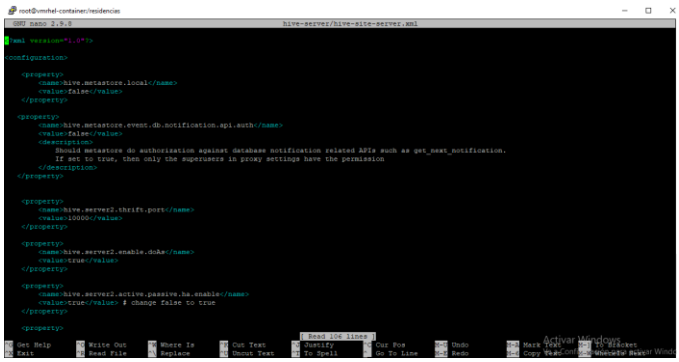

Figura 10.- Configuración hive site server

19.- Creamos archivo de configuración core-site.xml para asignar memoria.

nano core-site.xml

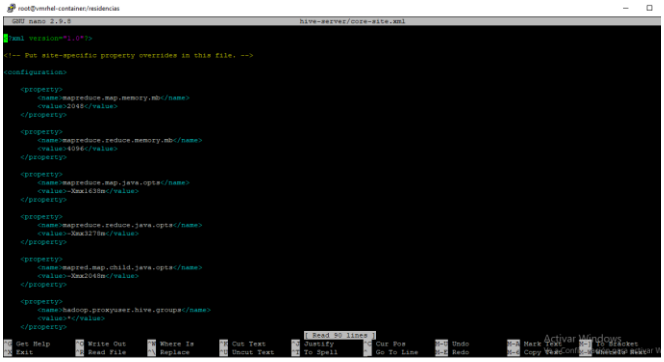

Figura 11.- Configuración core site server

20.- Creamos contenedor hive-server

podman run -it --name hive-server --hostname hive-server -p 10000:10000 -p 10002:10002 --privileged -v /residencias/hive-server/hive-site-server.xml:/opt/apache-hive-3.1.2-bin/conf/hive-site.xml -v /residencias/hive-server/core-site.xml:/opt/hadoop-3.3.1/etc/hadoop/core-site.xml -v /residencias/hive-server/hosts:/etc/hosts starburstdata/hive:3.1.2-e.15-x64 /bin/bash

21.- Accedemos al contenedor hive-server y ejecutaremos en segundo plano el servicio de para iniciar el proceso.

nohup ./hive --service hiveserver2 &

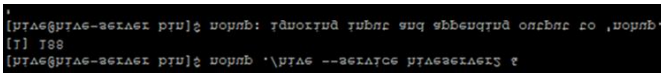

Figura 12.- Servicio hive server segundo plano

22.- Hacemos conexión a HiveServer2 mediante Beeline.

Apache Beeline es un cliente de Hive que se incluye en los nodos principales del clúster.

./beeline -u jdbc:hive2://hive-server:10000 -n hive -p hive

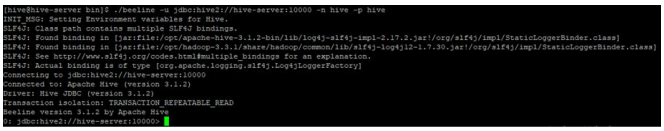

Figura 13.- Cliente beeline

Una vez la conexión sea exitosa se creara el schema de conexión y podremos ver los datos de minio ya almacenados en forma de metadatos en el metastore de hive que es postgreSQL.

Dentro del esquema creado por el metastore de hive en postgresql se crearon las referencias de los metadatos.

23.- Conectamos a la base de datos del metastore.

Con el comando “\c” nos podemos conectar a una base de datos.

\c test99;

24.- Consultamos las carpetas del schema creado.

select * from "SDS";

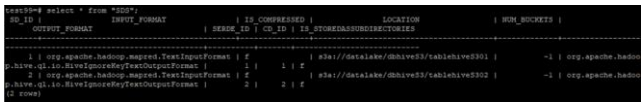

Figura 14.- Consulta al SDS de datos

select * from "DBS";

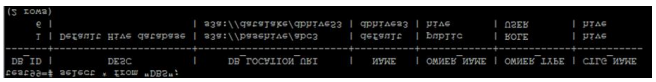

Figura 15.- Consulta al DBS de datos

select * from "TBLS";

Figura 16.- Consulta a TBLS de datos

Preparación contenedor Apache Atlas

25.- Descargamos imagen contenedor Apache Atlas

podman pull docker.io/wbaa/rokku-dev-apache-atlas.

26.- Creamos contenedor Apache Atlas

#podman run -it -d --name atlas-server -p 21000:21000 docker.io/wbaa/rokku-dev-apache-atlas

27.- Accedemos al contenedor para poder modificar archivos de configuracion para conexión con hive.

podman exec –it atlas-server /bin/bash

28.- Modificamos archivo apache-application.properties y agregamos la configuración de conexión hacia hive.

#Hive

#atlas.hook.hive.synchronous=false

#atlas.hook.hive.numRetries=3

#atlas.hook.hive.queueSize=10000

#atlas.cluster.name=primary

29.- Modificamos los archivos hive-site.xml de los contenedores hive y agregamos la siguiente propiedad para que pueda ser visible para atlas.

```
<property>
    <name>hive.exec.post.hooks</name>
    <value>org.apache.atlas.hive.hook.HiveHook</value>
  </property>
```

30.-Arrancamos el servicio de atlas accederemos por la web para poder acceder a la interfaz gráfica por el puerto correspondiente, por default usuario y passwor es “admin”.

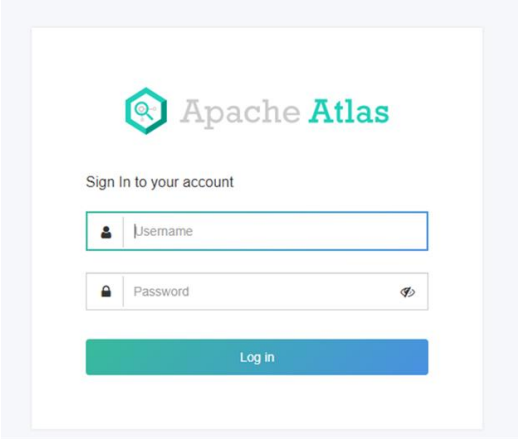

Figura 17.- Login Apache Atlas

Atlas prácticamente cuanta con un dashboard el cual permitirá realizar las búsquedas y creaciones de entidades, clasificaciones, glosarios y términos.

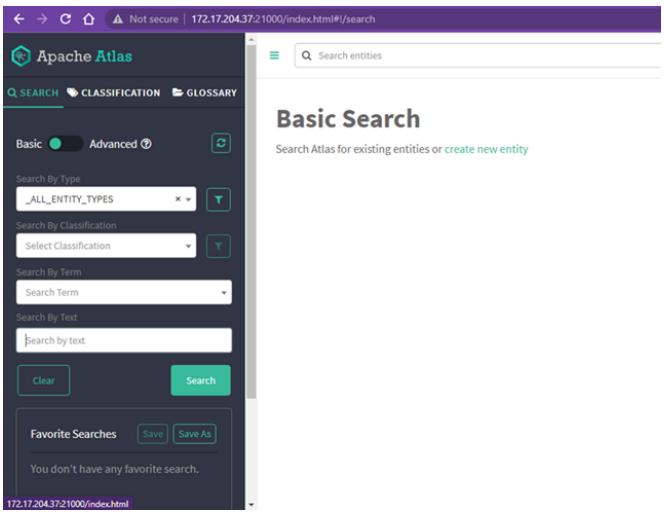

Figura 18.- Interfaz web apache atlas

Creación manual de linaje de datos apache atlas con formato JSON.

El linaje de datos explica cómo fluyen los datos en el sistema, como dónde se originan, qué transformaciones se le aplicaron, a dónde se movió, etc.

Para crear el líneage de datos necesitamos hacerlo en el lenguaje nativo que utiliza atlas, en este caso JSON y estar conectados a la red privada de INEGI para poder acceder a los servidores y extraer la información de minIO.

Conexión VPN para acceder al lago de datos de INEGI.

31.- Instalamos aplicación Cisco AnyConnect

Cisco AnyConnect es un agente unificado de terminales de seguridad que ofrece diversos servicios de seguridad para proteger la empresa. También proporciona la visibilidad y el control que necesita para identificar qué usuarios y dispositivos acceden a la empresa extendida.

Con esta aplicación podremos agregar la red de INEGI y podremos tener acceso al lago de datos, con el uso de las credenciales proporcionadas por los asesores.

32.- Ejecutamos aplicación con el siguiente comando.

vpn connect server_add group_name user_name passwd

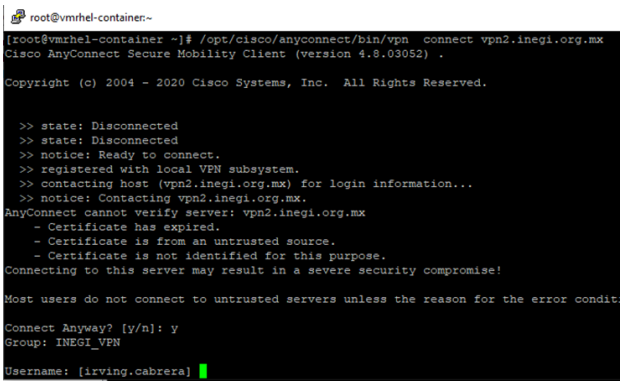

Figura 19.- Conexión vpn inegi

Instalación minio client

Este cliente nos permitirá realizar las mismas funciones que tenemos en la interfaz web de minio, pero con la diferencia de que podremos extraer o hacer consultas con los formatos que necesitamos, en este caso en formato JSON, que es la forma nativa en que apache atlas manipula los datos.

33.- Descargamos mc para minio

curl https://dl.min.io/client/mc/release/linux-amd64/mc --create-dirs -o $HOME/minio-binaries/mc

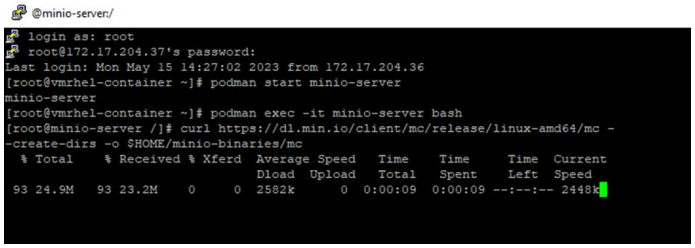

Figura 20.- Descarga mc

34.- Damos permisos a la carpeta de mc.

#chmod +x $HOME/minio-binaries/mc

35.- Agregamos la ruta al path del sistema.

#export PATH=$PATH:$HOME/minio-binaries/

36.- Creamos un alias con la cual hará referencia a la instancia de minio y sus respectivas credenciales.

mc alias set myminio https://minioserver.example.net ACCESS_KEY SECRET KEY

```
[root@minio-server /]# mc admin info local
●  172.17.204.37:9000
   Uptime: 1 minute
   Version: 2023-01-31T02:24:19Z
   Network: 1/1 OK
   Drives: 1/1 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 1

36 B Used, 2 Buckets, 5 Objects
1 drive online, 0 drives offline
[root@minio-server /]#
```

Figura 21.- Información conexión al bucket

37.- Usaremos el comando mc stat para la consulta de metadatos de los objetos, este nos traerá la información en formato JSON.

mc --json stat ALIAS/bucket/…/…

```
[root@minio-server /]# mc --json stat local/datalake/dbhiveS3/tablehiveS302/000000_0
{
 "status": "success",
 "name": "000000_0",
 "lastModified": "2023-02-26T01:03:40Z",
 "size": 8,
 "etag": "5a3ac4da87f744d0235089417ae3d757",
 "type": "file",
 "metadata": {
  "Content-Type": "application/octet-stream"
 }
}
{
 "status": "success",
 "name": "000000_0_copy_1",
 "lastModified": "2023-02-26T01:15:46Z",
 "size": 9,
 "etag": "32e2a3ee9f5eba3bf608e45c377ace36",
 "type": "file",
 "metadata": {
  "Content-Type": "application/octet-stream"
 }
}
{
 "status": "success",
 "name": "000000_0_copy_2",
 "lastModified": "2023-02-27T17:59:50Z",
 "size": 11,
 "etag": "9b7e586604005887f98ab94f845e27c91",
 "type": "file",
 "metadata": {
  "Content-Type": "application/octet-stream"
 }
}
{
 "status": "success",
```

Figura 22.- Consulta de metadatos formato json

Entidades, relaciones y línea de datos forma manual

En apache atlas se deberá hacer referencia a una entidad, que será el nodo que representara cualquier tipo de datos, en atlas cada entidad deberá ser con respecto a su tipo de datos y estas pueden tener entradas y salidas, se pueden representar nodos que hagan referencia a un proceso donde va a dar otro dato como salida.

38.- Definimos las entidades en la estructura de atlas.

Primero creamos todos los archivos json

```
-rw-r--r--. 1 root root     966 May 25 11:33  ENOE1mapReduce.json
-rw-r--r--. 1 root root     413 May 25 10:03  ENOE_EDAD15.json
-rw-r--r--. 1 root root     865 May 25 11:47  ENOEeventosinfomapReduceProcessInstance.json
-rw-r--r--. 1 root root     915 May 25 10:44  ENOEmapAndReduceProcess.json
-rw-r--r--. 1 root root     754 May 25 11:23  ENOEmapReduceProcessInstance.json
-rw-r--r--. 1 root root     903 May 25 10:44  ENOEmergerProcess.json
-rw-r--r--. 1 root root     909 May 25 12:08  ENOEtemasmapReduceProcessInstance.json
-rw-r--r--. 1 root root     228 May 25 10:35  eventos_info.json
-rw-r--r--. 1 root root     263 May 25 10:03  eventos_informacion_cuestionario.json
-rw-r--r--. 1 root root     408 May 25 10:03  eventos_informacion.json
-rw-r--r--. 1 root root     351 May 25 10:03  eventos_informacion_proyecto.json
```

Figura 23.- Archivos formato json

39.- Con el comando curl y referencia a la api haremos uso del método POST para crear las entidades.

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @ENOE_EDAD15.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @proveedor_datos.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @eventos_info.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @eventos_informacion_proyecto.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @eventos_informacion_cuestionario.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @temas_incluye.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @tema_1.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @tema_2.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @tema_3.json

*nota:cada que se crea una entidad apache atlas crea un guid de seguimiento de las entidades, este deberá de ser guardado para posteriormente usarlo al crear las relaciones entre entidades.

40.- Almacenar guid entidades

ENOE_EDAD15.json ,"guid":"1b79e829-b1fb-41b7-863d-e7612cad8a73"

proveedor_datos.json ,"guid":"b71873f5-21cd-46e5-91e3-3340b07e7e3d"

eventos_info.json "guid":"54b47cf7-92fc-48cc-a080-1d1e3730577a"

eventos_informacion_proyecto.json "guid":"13a761b7-5997-423c-9b39-f36579f701cf"

eventos_informacion_cuestionario.json,"guid":"b12defd8-0696-4ad3-9f3c-9f83b3c27eb8"

temas_incluye.json ,"guid":"f176c884-3ef7-4d8a-b274-97f447fcc897"

tema_1.json ,"guid":"bb77adee-caa8-4560-8aa3-2a83242d7ccf"

tema_2.json "guid":"c35aa0b9-21a0-4859-9499-cf2b22963a75"

tema_3.json "guid":"b73a11ce-88bd-4639-a192-f574158e0c39"

41.- Definimos las entidades tipo proceso que serán las que van a tener las relaciones.

nano ENOEmergerProcess.json

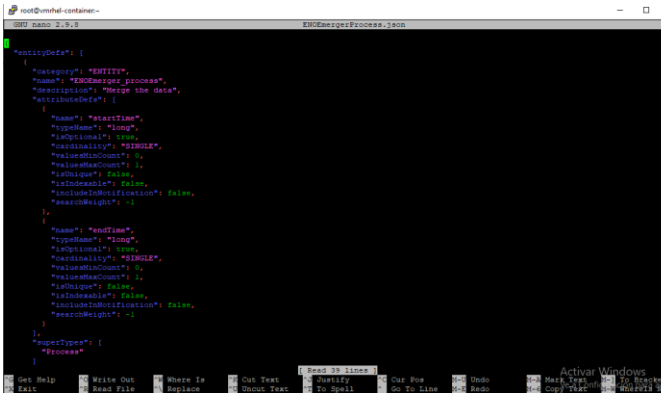

Figura 24.- Json entidad proceso merger

nano ENOEmapAndReduceProcess.json

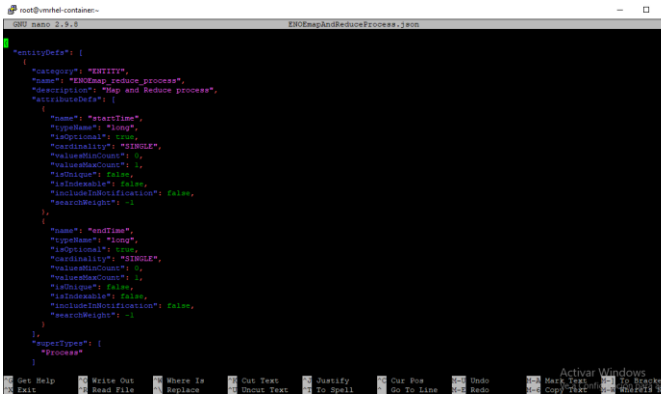

Figura 25.- Json entidad proceso mapandreduce

42.- Con el comando curl crearemos le tipo de proceso

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/types/typedefs -d @ENOEmapAndReduceProcess.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/types/typedefs -d @ENOEmergerProcess.json

Guid entidades prceso

ENOEmergerProcess.json ,"guid":"e18a86c0-a4a1-4d6c-b6df-b80cf8edbff1"

ENOEmapAndReduceProcess.json "guid":"fcb7f99f-e911-4509-b569-3ffb93c18d5d"

43.- Creamos relaciones entre entidades

Relación 1

nano ENOEmapReduceProcessInstance.json

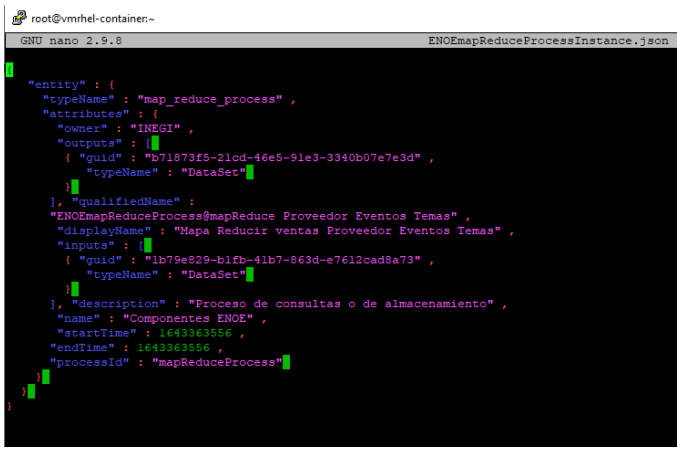

Figura 26.- ENOEmapReduceProcessInstance.json

Relación 2

nano ENOEeventosinfomapReduceProcessInstance.json

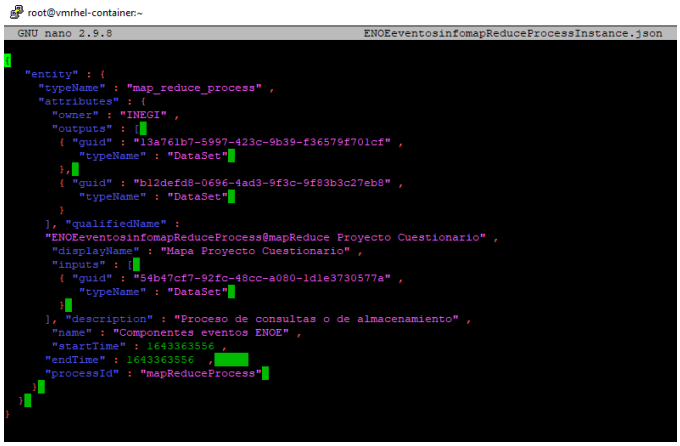

Figura 27.- ENOEeventosinfomapReduceProcessInstance.json

Relación 3

nano ENOEtemasmapReduceProcessInstance.json

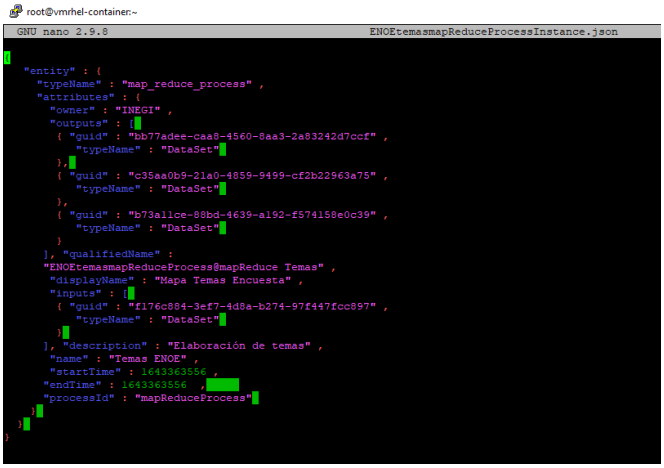

Figura 28.- ENOEtemasmapReduceProcessInstance.json

44.- Con el comando curl y referencia a la api haremos uso del método POST para crear las instancias.

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @ENOEmapReduceProcessInstance.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @ENOEeventosinfomapReduceProcessInstance.json

curl -X POST -u admin:admin -H 'accept: application/json' -H 'cache-control: no-cache' -H 'content-type: application/json' http://localhost:21000/api/atlas/v2/entity -d @ENOEtemasmapReduceProcessInstance.json

De esta manera podemos ir visualizando como es que se generan las entidades y el grafo con las relaciones que estamos asignando.

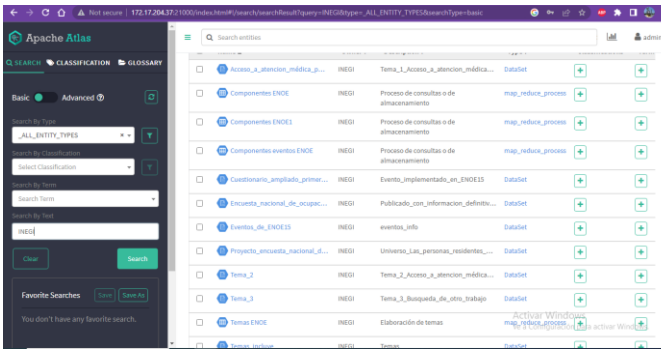

Figura 29.- Interfaz búsqueda entidades

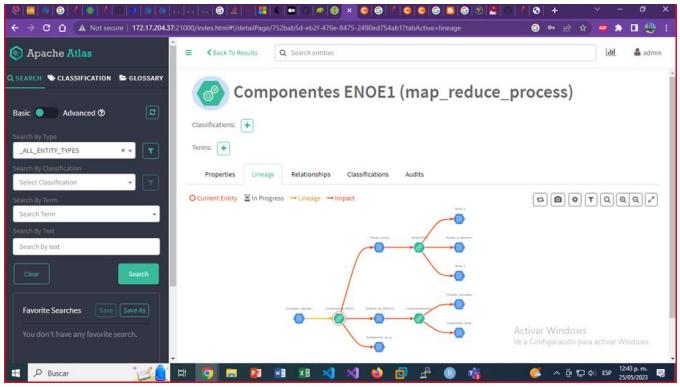

Figura 30.- Interfaz web y diseñado

Ahora podemos ver el grafo del prototipo del dataset analizado.

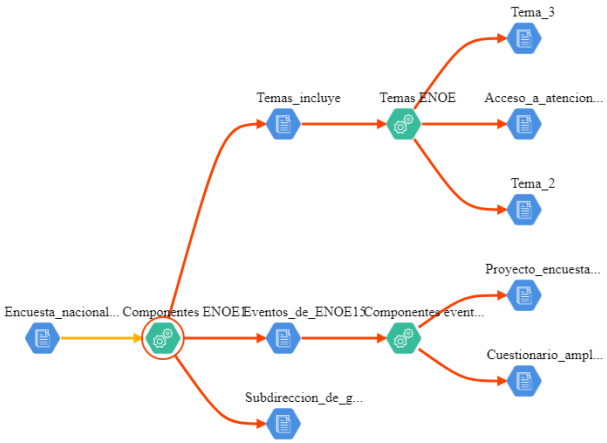

Figura 31.- Lineage de datos

III. CONCLUSIONES

Con este proyecto se pusieron en práctica nuevos conocimientos, desde hacer uso de un sistema distinto, con una línea de aprendizaje más complejo y hacer uso de aplicaciones que están dentro del concepto de big data, esto requirió que se hiciera una profunda investigación para poder entender los conceptos y saber cómo funciona cada una de las herramientas para así poder realizar la solución adecuada. El catálogo de datos involucra muchos conceptos de conocimiento, desde saber sistemas operativos, conceptos de almacenamientos, redes, cómputo en la nube, almacenamientos, análisis de datos,

Aparte con el uso del sistema en sus mínimos requerimientos (modo consola) nos permitió desarrollar más conocimiento acerca del sistema RHEL, haciendo que todo fuera de forma más manual y esto nos permitió darnos cuenta de todo los pasos y comandos que son necesarios para que funcione el sistema, normalmente no nos damos cuenta por utilizar sistemas automatizados donde solo hay que dar siguiente y esto nos limita

a saber de dónde pueden surgir los errores a la hora de estar en ejecución.

En mi experiencia la verdad fue un reto totalmente nuevo y complejo, se tuvieron que aprender muchas cosas, aparte de hacer uso de un sistema al cual no estaba familiarizado, esto me hizo verme en la necesidad de investigar y generar nuevo conocimiento, aplicándolo aprendiendo a base de pruebas y errores, probando distintos métodos hasta llegar al más óptimo y que se adecuo a las aplicaciones que se utilizan dentro del INEGI.

AGRADECIMIENTOS

Principalmente quiero agradecer a la institución por brindarme la oportunidad de colaborar y realizar mis residencias profesionales, las mismas que me han abierto el panorama a mas conocimiento y obstáculos, los cuales me han ayudado a desarrollarme dentro de un ambiente profesional que me orillo a resolver problemas, buscar alternativas, tomar decisiones, y así poder encontrar soluciones para poder cumplir con el objetivo del proyecto.

Quiero agradecer a mis asesores por brindarme su disponibilidad y apoyo para cualquier situación.

Quiero agradecer a mis padres por brindarme su apoyo en todo momento.

REFERÊNCIAS BIBLIOGRÁFICA

1.- ¿Qué son los metadatos y cuál es su utilidad? (n.d.). https://blog.powerdata.es/el-valor-de-la-gestion-de-datos/que-son-los-metadatos-y-cual-es-su-utilidad

2.- ¿Qué es un catálogo de datos? (n.d.). TIBCO Software. https://www.tibco.com/es/reference-center/what-is-a-data-catalog#:~:text=El%20cat%C3%A1logo%20consiste%20principalmente%20en,tendr%C3%A1%20un%20cat%C3%A1logo%20de%20datos.

3.- García, Ó., & García, Ó. (2022, May 26). Data Governance usando Apache Atlas - Damavis Blog. Damavis Blog - Data - Machine Learning - Visualization. https://blog.damavis.com/data-governance-usando-apache-atlas/

4.- Morales, F., & Morales, F. (2022b). Almacenamiento como servicio (minIO) | Sysadmins de Cuba. Sysadmins De Cuba. https://www.sysadminsdecuba.com/2022/02/almacenamiento-como-servicio-minio/

5.- Borges, S., & Borges, S. (2019). Servidor PostgreSQL. Infranetworking. https://blog.infranetworking.com/servidor-postgresql/

6.- Oscarfmdc. (2023). Apache Hive: Introducción. Aprender BIG DATA. https://aprenderbigdata.com/apache-hive/

7.- HiveServer2 Overview - Apache Hive - Apache Software Foundation. (n.d.). https://cwiki.apache.org/confluence/display/Hive/HiveServer2+Overview

8.- Design - Apache Hive - Apache Software Foundation. (n.d.). https://cwiki.apache.org/confluence/display/hive/design#Design-Metastore

9.- ¿Qué es y para qué sirve la virtualización? (n.d.). https://www.redhat.com/es/topics/virtualization/what-is-virtualization

10.- Sistema operativo Red Hat Enterprise Linux. (n.d.). https://www.redhat.com/es/technologies/linux-platforms/enterprise-linux

11.- En qué consiste la contenerización | IBM. (n.d.). https://www.ibm.com/es-es/topics/containerization#:~:text=el%20siguiente%20paso-,%C2%BFQu%C3%A9%20es%20la%20contenerizaci%C3%B3n%3F,manera%20coherente%20en%20cualquier%20infraestructura

12.- Podman: el motor de contenedores sin daemon. (2023, January 3). IONOS Digital Guide. https://www.ionos.mx/digitalguide/servidores/herramientas/tutorial-de-podman/